A Faster FPRAS for #Knapsack

Liran Markin

August 2026

Abstract

The fastest known approximation scheme for counting knapsack solutions is the $\widetilde{O}(n^{1.5}\varepsilon^{-2})$ FPRAS of Feng and Jin (SODA 2025). We observe that the arrays their construction stores are, exactly, a product-form factorization of the distribution their sampler needs, and that sampling along this factorization – instead of scanning arrays – drops the total running time to $\widetilde{O}(n^{1.5} + \min(n\sqrt{\ell}, n^2/\ell) \cdot \varepsilon^{-2}) \le \widetilde{O}(n^{1.5} + n^{4/3}\varepsilon^{-2})$: never worse, and strictly faster for every $\varepsilon = o(1)$.[1]

1 The problem and the result

#Knapsack asks: given item weights $W_1, \ldots, W_n$ and a capacity T, how many of the 2^n subsets have total weight at most T? The problem is #P-hard, so one settles for a randomized $(1 \pm \varepsilon)$-approximation (an FPRAS). After two decades of progress (Dyer's $\widetilde{O}(n^{2.5}\varepsilon^{-2})$ [2] through several deterministic and randomized improvements), the record is the $\widetilde{O}(n^{1.5}\varepsilon^{-2})$ algorithm of Feng and Jin [1].

Theorem 1. *The Feng–Jin FPRAS can be implemented in time*

$$\widetilde{O}\Big(n^{1.5} \;+\; \min\Big(n\sqrt{\ell},\; \tfrac{n^2}{\ell}\Big) \cdot \varepsilon^{-2}\Big) \;\le\; \widetilde{O}\Big(n^{1.5} + n^{4/3}\,\varepsilon^{-2}\Big)\,,$$

where $\ell \in [2, 8n]$ is the instance's popular weight-class parameter ([1], Lemma 3.1).

At $\varepsilon = \Theta(1)$ the two bounds coincide at $n^{1.5}$; the moment ε shrinks, the ε^{-2} term dominates and the coefficient drops from $n^{1.5}$ to at most $n^{4/3}$. For scale: at $\varepsilon = 1/\sqrt{n}$, their bound reads $n^{2.5}$, ours $n^{11/6}$.

2 Where their time goes

Feng–Jin follow Dyer's recipe: count a rounded, easier instance exactly-ish, then correct the count by estimating, via random sampling, the fraction of rounded solutions that are true solutions. Both halves run on the same data structure: a divide-and-conquer tree over the items, where each node stores an array approximating the counting function of its sub-instance (array$[s] \approx$ number of sub-solutions of rounded weight s).

[1] A Lean 4 development accompanying this memo machine-checks the components introduced here; see the repository.

Construction of all arrays costs $\widetilde{O}(n^{1.5})$. The expensive half is *sampling*: drawing one uniform random rounded solution means walking the tree from the root, and at every node splitting the remaining weight between the two children with probability proportional to a product of the children's array entries. Their sampler draws this split by scanning the arrays – length $\widetilde{O}(\ell)$ at the top of a class tree – so each of the $N = \widetilde{O}(n/(\ell\varepsilon^2))$ samples costs $\widetilde{O}(\ell\sqrt{m})$, and the sampling stage totals $\widetilde{O}(n^{1.5}\varepsilon^{-2})$. That scan is the entire ε^{-2} bottleneck.

3 The observation

The improvement comes from looking closely at the *representation* Feng–Jin already use. Array entries are counts as large as 2^n, so to multiply arrays in subquadratic time they store every entry in the form $D^{\text{level}} \cdot \text{residual}$ with the residual in $[1, D)$, $\log D = \text{polylog}$, and merge two children with a bounded $(\max, +)$-convolution that also returns *witness counts* (their Theorem 6.1): for each output position, the total mass of the level-sum-*maximal* pairs.

Unwinding their definitions, the merged value stored at position s satisfies, exactly,

$$h(s) \;=\; \sum_{\substack{y+z=s \\ \text{level-sum attaining}}} f_L(y)\, f_R(z),$$

because on every witness pair the level scaling cancels: $D^C \cdot u(y)v(z) = f_L(y)f_R(z)$. This is an identity, not an approximation. In words:

> *The arrays already store a product-form factorization of the distribution the sampler needs. The witness machinery was built to compute the arrays fast; as an unused by-product, it makes them samplable fast.*

A sampler that draws each split proportionally to this stored factorization therefore introduces *no additional error at all* – the tree of arrays defines a distribution, and we sample exactly it, so the Feng–Jin correctness analysis (which compares that distribution to the uniform one) applies verbatim.

4 The sampler

One sample proceeds as follows.

1. **Root draw.** Draw the total rounded weight $s \propto \hat{f}_{\text{root}}(s)$ over $s \le T$, by one binary search in a precomputed prefix table. This is the *only* place the capacity constraint appears.
2. **De-rounding.** Positions are re-rounded to coarser grids up the tree. Rounding is a monotone floor, so each coarse position's preimage is a contiguous interval of the finer grid; drawing the fine position within it is another exact interval draw. Consequently every recursive step below receives an *exact* child weight – no differences of approximate quantities ever arise.
3. **Split draw.** At a node with exact position s, draw an attaining pair $(y,\ s-y)$ with probability $\propto u(y)v(s-y)$. The attaining pairs organize into *level rectangles* – pairs of intervals with a fixed level sum – and there are only $M = \widetilde{O}(\text{subtree items/polylog})$ of them, far fewer than the array length near the root, which is exactly where the old scans were expensive. Within

a rectangle the residuals span a polylog range, so a factor-2 bucketing of values plus precomputed indicator convolutions (all shifts at once, one FFT per node, inside the construction budget) supports an exact draw with at most four expected rejections.

4. **Pruning.** If a child's drawn weight is 0 and its only weight-0 subset is the empty set, emit $\varnothing$ without descending. This is distribution-exact, and it means a sample that contains k items visits only the $\widetilde{O}(k)$ tree paths of those items.

5 Why this is $n^{4/3}$

Per sample, at depth h of a weight-class tree, the pruned walk visits $\min(2^h, k)$ nodes, and each split draw costs $\min(\text{items}/2^h,\ \ell/2^{h/2})$ – the rectangle count capped by the plain scan (the new method is never slower than the old one). Summing over depths,

$$\text{per sample} \;=\; \widetilde{O}\big(\min(n_m,\ \ell\sqrt{k})\big), \qquad k = \widetilde{O}(\ell),$$

and multiplying by the sample count $N = \widetilde{O}(n/(\ell\varepsilon^2))$:

$$N \cdot \text{per sample} \;=\; \widetilde{O}\Big(\min\Big(\tfrac{n^2}{\ell},\ n\sqrt{\ell}\Big)\,\varepsilon^{-2}\Big).$$

The two branches cross at $\ell = n^{2/3}$, where the value is $n^{4/3}\varepsilon^{-2}$ – the worst case of Theorem 1. At the extremes ($\ell =$ polylog or $\ell = \Theta(n)$) the bound improves further, to $\widetilde{O}(n^{1.5} + n\,\varepsilon^{-2})$.

6 A further refinement: amortized caching

The per-draw rectangle enumeration can be amortized. Two observations compound. First, the position queried at a node is an index into that node's array, so at most L_u distinct (node, position) pairs ever occur – however often the node is visited; caching an alias table per pair makes repeat draws free. Second, a build enumerates at most $\min(M_u, L_u)$ rectangles, not M_u: every witness level present in the array occupies at least one of its L_u cells. Summed over a class tree, all cache builds together cost $\widetilde{O}(\ell^2)$ – free of both ε and n. A third step goes further: precompute the witness merge on *dyadic sub-blocks* of a child, deepening a node only as its visits double. The exact identity "a block contributes its own witness mass when its block-maximum matches the global maximum, and nothing otherwise" lets a draw binary-search the stored block weights, and the doubling schedule balances build against draw at $L_u\sqrt{\text{visits}}$ per node, i.e. $\widetilde{O}(\ell\sqrt{Nk})$ in total. Taking the best mode at each ℓ:

$$\widetilde{O}\Big(n^{1.5} + \min\big(n^{5/4}\varepsilon^{-3/2},\ n^2\big) + n\,\varepsilon^{-2}\Big).$$

This is strictly below Theorem 1's $n^{4/3}\varepsilon^{-2}$ for *every* $\varepsilon < 1$, stays flat at $\widetilde{O}(n^{1.5})$ for all $\varepsilon \geq n^{-1/6}$, and reaches the output-optimal $n\varepsilon^{-2}$ once $\varepsilon \leq n^{-1/2}$. The remaining ε-term sits exactly at the doubling balance; answering a fresh position's witness query below $L\sqrt{\text{visits}}$ is the open subproblem this refinement exposes.

7 Discussion

Two remarks. First, the bound is *instance-adaptive*: ℓ is computable in $\widetilde{O}(n)$ time, so the algorithm knows its own speed in advance, and only a narrow band of instances around $\ell \approx n^{2/3}$ realizes the

worst case. Second, the remaining bottleneck is transparent: the per-draw rectangle enumeration. Removing it – sampling a rectangle in polylog time – would give $\widetilde{O}(n^{1.5} + n\,\varepsilon^{-2})$ uniformly, and any further progress below $n^{1.5}$ is blocked by the bounded monotone $(\max, +)$-convolution frontier, matching the open problems stated in [1].

References

[1] W. Feng, C. Jin. *A Faster Algorithm for Counting Knapsack Solutions*. SODA 2025. arXiv:2410.22267.

[2] M. Dyer. *Approximate counting by dynamic programming*. STOC 2003.

[3] P. Gawrychowski, L. Markin, O. Weimann. *A Faster FPTAS for #Knapsack*. ICALP 2018.